

Design and Analysis of FFT for Signal Processing Application

¹S.Indira and ²K.Mariamammal

Madras Institute of Technology, Anna University

Chrompet, Chennai-44

¹indirasuresh20996@gmail.com, ²mariammalkms@gmail.com

Abstract- This paper presents a modified Radix-4 Fast Fourier Transform (FFT) signal flow graph whose input and output signal are in same order. Conventional radix-4 FFT which has input signal in same order but output has mismatch order. An output is in mismatch order, it increases computation time. Memory based modified FFT processor avoids address conflict. Compared to conventional radix-4 FFT, the proposed memory based modified FFT processor has minimum computation time under the lesser resource consumption. It decreases area, power and computation time.

Index term- Fast Fourier Transform (FFT), Discrete Fourier Transform (DFT), Signal Flow Graph (SFG), Real Datapath Fast Fourier Transform (RFFT) and Digital Signal Processing (DSP).

1. INTRODUCTION

Fast Fourier Transform (FFT) plays a major era in digital signal processing. FFT is an algorithm that computes DFT of sequence. Fourier analysis convert signal from its original domain (often time or space) to representing in frequency domain and vice versa. Discrete Fourier Transform (DFT) convert finite sequence of equally spaced sample of a function into same length sequence of equally spaced sample of discrete time Fourier Transform. Normally DFT computation order require N^2 operation. FFT reduces the order of computation $N \log_2 N$. FFT decompose the set of data to be transformed into series of smaller data set to be transformed then it decomposes those smaller set into ever smaller set. At each stage the result of processing, the result of previous stage is combined with twiddle factor

multiplication. Finally, FFT calculate at each smaller data set.

Most of signal processing application require FFT processor which has input and output signal both in same order. Conventional radix-4 FFT signal flow graph which input in natural order and output in mismatch order. In order to realize output in natural order it increase computation time. A modified radix-4 FFT signal flow graph both input and output in same order. It avoids mismatch order and its decreases computation time.

Two kinds of architecture, there are pipelined and memory architecture. Pipelined architecture increases throughput and performance but it has more resource consumption. Since, it uses independent processing element in each stage. Memory architecture has lower throughput, but it has smaller resource consumption and power. Since, it reuses the same processing element in all stage. Memory based modified FFT avoids address conflict and corresponding memory-addressing scheme.

Memory based architecture reuse same butterflies and rotator to the next stage [1]-[5]. Pipelined architecture is used to reduced memory requirement from N to $N/2$ [2]. By using time division multiplexer to reduced arithmetic operation to achieve 100% utilization ratio [3]. Data permutation method avoid bit reversal order [4]. A Constant geometry achieves balance iteration cycle using maximum frequency [6]-[9]. Decomposition algorithm method to decrease computation cycle [7]. Split radix FFT algorithm significantly decrease as the FFT size increase [8]. RFFT architecture support continuous flow operation and normal order output [10]. Pipelined architecture is used to

increases throughput and overall performance of the CPU. It is suitable for real time application [11]. DSP allows each parallel processor to execute every four instruction per cycle [12]. Memory customization approach are applied both pipelined and memory-based [13]. To reduced the data word length for FFT and decrease the energy consumption [14]. Based on hashing and lookup table technique to reduce number of arithmetic operations required to perform FFT [15].

Section II describes the radix-4 from conventional and modified signal flow graph. Section III discusses about the memory-based modified radix-4 FFT. Section IV discusses about the implementation result and comparison. Section V discusses about conclusion and future scope.

II. RADIX-4 SIGNAL FLOW GRAPH

Radix-4 FFT computation is discussed in this section which includes conventional 16-point and modified 16-point FFT.

A. Radix-4 FFT Algorithm: The N-point DFT of an input sequence $X(K)$ in equation (1) and twiddle factor in equation(2)

$$X(K) = \sum_{n=0}^{N-1} x(n) W_N^{kn} \quad (1)$$

$$W_N^{kn} = e^{-j2\pi kn/N} \quad (2)$$

When N is a power of two. The FFT based on the Cooley –Tukey algorithm is main algorithm in order to compute the DFT efficiently. The Cooley-Tukey algorithm reduces the number of operations N^2 for the DFT to $N \log_2 N$ for the FFT. To calculate FFT in series of $n = \log_p N$ stages where p is the base of the radix. Radix-4 is very efficient compared to radix-2 FFT.

The radix-4 FFT equation (3) is listed below

$$X(K) = \sum_{n=0}^{N-1} [x(n) + (-j)^k x(n+N/4) + (-1)^k x(n+N/2) + (j)^k x(n+3N/4)] W_N^{nk} \quad (3)$$

Radix-4 shown in Fig 1. The radix-4 FFT require two stage of a radix-2 FFT. To calculate number of stages in 16-point FFT, the radix-4 has $\log_4 16 = 2$ stage and radix-2 has $\log_2 16 = 4$ stages. It reduces no of adder

and multiplication in each stage and also it reduces no of stages.

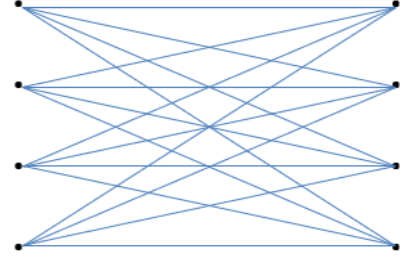


Fig 1 Radix-4 FFT

B. Conventional Radix-4 Signal Flow Graph

Conventional 16-point radix-4 FFT signal flow graph has contain two stages. 16-point radix-4 computes four input into four output in first stage and multiplied with twiddle factors to obtained result in each stage. Conventional signal flow graph is shown in Fig-2.

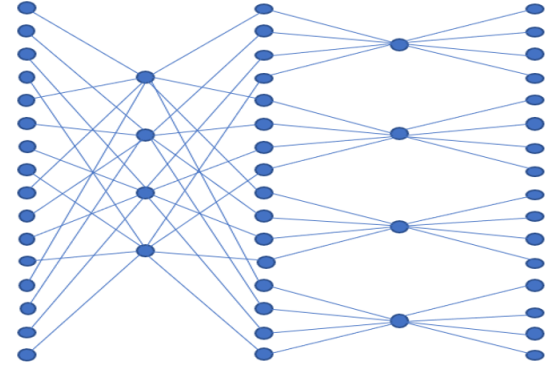


Fig-2 Conventional 16-point radix-4 FFT

C. Modified Radix-4 Signal Flow Graph

Modified 16-point radix-4 FFT signal flow graph contain two stages. It has input and output both in natural order. So, it avoids bit reverse operation and it decreases the computation time effectively. Fig 3 shows that modified 16-point radix-4 FFT.

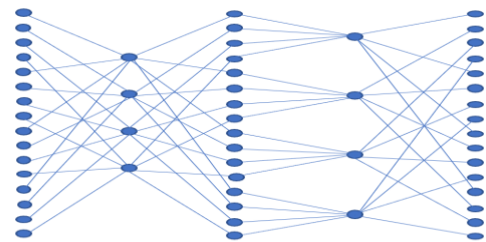


Fig 3 Modified 16-point radix-4 FFT

III MEMORY-BASED FFT ARCHITECTURE

A. FFT Architecture

An architecture of FFT processor contain two memory port (ping and pang), 16-point FFT divides into four banks, two stages radix-4 computation, two multiplexer networks to fetches the address for each module in FFT. The proposed FFT architecture shown in fig-4.

The memories of each stage FFT are used to store the output data of butterfly computation unit. Ping bank is used for generating four inputs of the butterfly unit and pang bank is used for receiving four results of FFT unit. Using multiplexer to fetches the address. Depending upon the bank arrangements to fetch address in modified signal flow graph. Using memory-based modified 16-point radix-4 for data storing and fetching address.

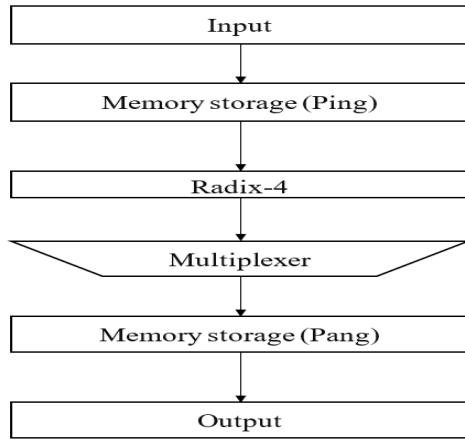


Fig 4 Radix-4 FFT architecture

B. Memory-based modified radix-4 FFT

16-point radix-4 FFT contain two stage in FFT processor. 16-point divides into four banks. Each bank contains four points in FFT.

The First stage of FFT: An input data is stored in sequence into the four banks. Point 1 is stored in address 1 of bank 1, Point 4 is stored in address 1 of bank 2, Point 8 is stored in address 1 of bank 3 and point 12 is stored in address 1 of bank 4. After all the input data are

stored, four-point computes first stage of radix-4 computation. Point 1, point 4, point 8 and point 12 are four output value simultaneously gets from the first clock cycle to the radix-4 computation unit. All the storage of other butterfly point is outputted in first stage. Fig 5 shows that first stage of butterfly computation.

Table 1 List of Memory address of each bank

BANK 0	BANK 1	BANK 2	BANK 3
4'b0000	4'b0100	4'b1000	4'b1100
4'b0001	4'b0101	4'b1001	4'b1101
4'b0010	4'b0110	4'b1010	4'b1110
4'b0011	4'b0111	4'b1011	4'b1111

The four butterfly computation results are generated at the four-computation cycle.

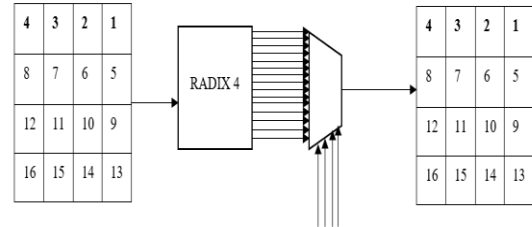


Fig 5 First stage of 16-point FFT computation

And the first result 1-1 of the butterfly computation are stored in first address of bank1 at the first clock cycle, and second result 1-2 of the butterfly computation 1 are stored in first address of bank2 at the second clock cycle and so on.

Table 2 First stages of butterfly computation

ADDRESS	SELECT	BANK 1	BANK 2	BANK 3	BANK 4
2'b00	00	1_0	1_1	1_2	1_3
2'b01	01	2_4	2_5	2_6	2_7
2'b10	10	3_8	3_9	3_10	3_11
2'b11	11	4_12	4_13	4_14	4_15

Table 2 listed the first stages of butterfly computation.

2) The Second stage of FFT: For the second stages of the FFT computation is same as the first stage. Point 1 is stored in address 1 of bank1, point 4 is stored in address 1 of bank 2, Point 8 is stored in address 1 of bank 3 and point 12 is stored in address 1 of bank 4. After all the input data are stored, four-point computes first stage of radix-4 computation. Point 1, point 4, point 8 and point 12 are outputted simultaneously at the first clock cycle from the first address of the four banks to the radix-4 computation unit. But multiplexer fetching address depending upon the modified radix-4 FFT. All the storage of other butterfly point is outputted in first stage. Fig 6 shows that second stage of butterfly computation.

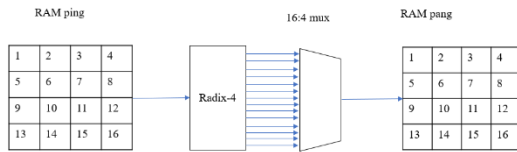


Fig 6 Second stage of butterfly computation

The four butterfly computation results 1-1, 2-1, 3-1 and 4-1 are generated at the four-computation cycle. And the first result 1-1 of the butterfly computation 1 are stored in first address of bank1, and second result 2-1 of the butterfly computation are stored in second address of bank1 and third result 3-1 of the butterfly computation are stored in third address of bank1 and so on.

Table 3 listed second stage of butterfly unit

ADDRESS	SELECT	BANK 1	BANK 2	BANK 3	BANK 4
2'b00	00	1_0	1_4	1_8	1_12
2'b01	01	2_1	2_5	2_9	2_13
2'b10	10	3_2	3_6	3_10	3_14
2'b11	11	4_3	4_7	4_11	4_15

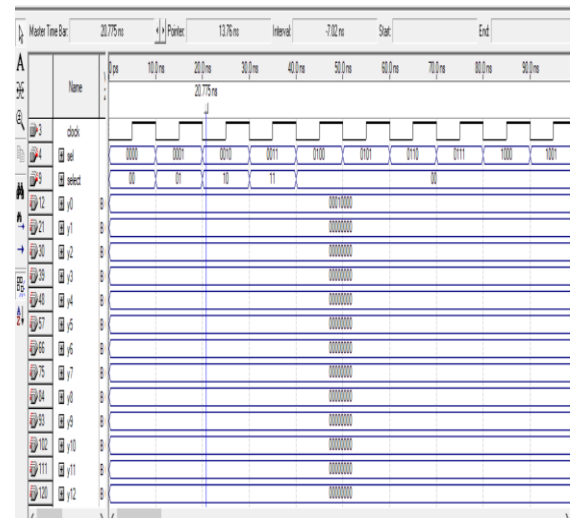
An output is in natural order, it decreases computation time under the lower resource consumption.

IV RESULTS AND DISCUSSION

In this work, Verilog HDL code is written and simulated using QUARTUS II 9.0 version. Stratix III family with the device EP35L50F484C2 is used to get synthesized results

A) Memory-based Modified 16-point radix-4 FFT

Fig 7 shows the simulated waveform for memory based modified signal flow graph 16-point FFT with an input data of Radix-4 FFT is $x(n) \{1,1,1,1,1,1,1,1,1,1,1,1,1,1\}$ and it computing each stage of Radix-4 FFT which gives a result of $X(k) \{16,0,0,0,0,0,0,0,0,0,0,0,0,0\}$. Fig 8 shows the simulated waveform of output data of memory based modified signal flow graph 16-point FFT.



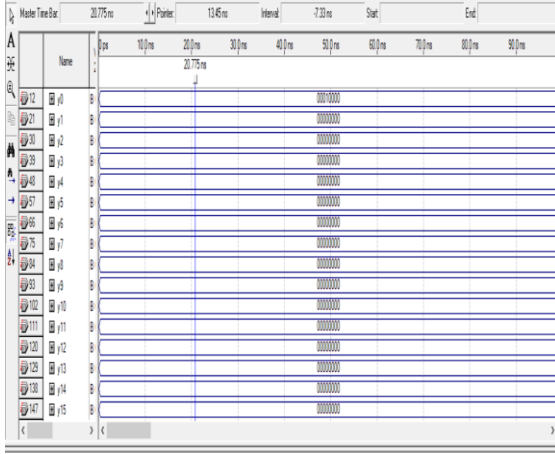


Table 4 synthesis summary

Parameter	Conventional 16-point FFT	Memory based Modified 16-point FFT
Area (logic elements)	289	34
Power(mW)	416	406
Delay(ns)	21.237	9.252

Table 4 list the synthesis summary of the conventional 16-point FFT and memory based modified 16-point FFT.

The memory-based modified 16-point radix-4 FFT processor has better overall result compared with area (logic element), power and computation time.

V. CONCLUSION

This work discusses the design of memory based FFT architecture. This memory based FFT architecture comprises of two memory storage namely ping and pang, two multiplexers and two stages of 16-point radix-4 FFT structure. The input for this architecture is in natural order and is stored in ping. The multiplexer is used to fetch the address from ping and then radix-4 16-point FFT computation is carried out. Then the computed radix-4 output is stored in pang. Then the radix-4 16-point FFT second stage computation is carried out. Then the next multiplexer is used to select the address from the pang in order to obtain the required natural order output data. The radix-4 16-point FFT contain two stages. The twiddle factors are

calculated and is used in second stage. This architecture is compared with the conventional 16-point FFT. The synthesis results are obtained using QUARTUS II 9.0 version. Stratix III family with the device EP35L50F484C2 and the results are reported.

The area (logic elements) of the designed memory based 16-point FFT architecture offers 88% area reduction and memory based 16-point FFT architecture offer 2% power reduction and memory based 16-point FFT architecture offer 56% timing reduction.

The future scope of this work is to implement radix-4 FFT with higher number of points and apply this architecture in digital filtering applications and to analyse the performance parameters.

REFERENCES

1. YinghuiTian , Yong Hei, Zhizhe Liu, Qi Shen, Zhixiong Di, and Tao Chen, "A Modified Signal Flow Graph and Corresponding Conflict-Free Strategy for Memory-Based FFT Processor Design". IEEE Trans. Circuit syst. II Exp Breifs, vol. 66, no. 1, Jan.2019
2. J. Q. Liu, Q. J. Xing, X. B. Yin, X. B. Mao, and F. Yu, "Pipelined architecture for a radix-2 fast Walsh-Hadamard-Fourier transform algorithm," IEEE Trans. Circuits Syst. II, Exp. Briefs, vol. 62, no. 11, pp. 1083–1087, Nov. 2015.
3. X.-B. Yin, F. Yu, and Z.-G. Ma, "Resource-efficient pipelined architectures for radix-2 real-valued FFT with real data paths," IEEE Trans. Circuits Syst. II, Exp. Briefs, vol. 63, no. 8, pp. 803–807, Aug. 2016.
4. Z. Qian and M. Margala, "Low-power split-radix FFT processors using radix-2 butterfly units," IEEE Trans. Very Large Scale Integer. (VLSI)Syst., vol. 24, no. 9, pp. 3008–3012, Sep. 2016.
5. M. Garrido, S.-J. Huang, S.-G. Chen, and O. Gustafsson, "The serial commutator FFT," IEEE Trans. Circuits Syst. II, Exp. Briefs, vol. 63, no. 10, pp. 974–978, Oct. 2016
6. M. Garrido, M. A. Sanchez, M. L. López-Vallejo, and J. Grajal, "A 4096-point radix-4 memory-based FFT using DSP slices," IEEE Trans. Very Large Scale Integer. (VLSI) Syst., vol. 25, no. 1, pp. 375–379, Jan. 2017.

7. Y. H. Tian et al., "A memory-based FFT processor using modified signal flow graph with novel conflict-free address schemes," *IEICE Electron.Exp.*, vol. 14, no. 15, pp. 1–11, Aug. 2017.
8. K.-F. Xia, B. Wu, T. Xiong, and T.-C. Ye, "A memory-based FFT processor design with generalized efficient conflict-free address schemes," *IEEE Trans. Very Large Scale Integer. (VLSI) Syst.*, vol. 25, no. 6, pp. 1919–1929, Jun. 2017.
9. Z.-G. Ma, X.-B. Yin, and F. Yu, "A novel memory-based FFT architecture for real-valued signals based on a radix-2 decimation-in-frequency algorithm," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 62, no. 9, pp. 876–880, Sep. 2015.
10. X.-B. Mao, Z.-G. Ma, F. Yu, and Q.-J. Xing, "A continuous-flow memory-based architecture for real-valued FFT," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 64, no. 11, pp. 1352–1356, Nov. 2017.
11. V. Venkata Lakshmi Dadala, C. H. Satya Naresh, R. Anil Kumar, "Butterfly Design for RADIX-4K DIF FFT," *International Journal of Research in Computer and Communication Technology*, Vol 3, Issue 10, October – 2014
12. Charles Wu "Implementing the Radix-4 Decimation in Frequency (DIF) Fast Fourier Transform (FFT) Algorithm Using a TMS320C80 DSP," *Digital Signal Processing Solutions*. January 1998.
13. Gyuseong Kang, Woong Choi, and Jongsun Park, "Embedded DRAM-Based Memory Customization for Low-Cost FFT Processor Design" *IEEE Trans. Very Large Scale Integer. (VLSI) Syst.* September 9, 2017.
14. Yifan Bo, Renfeng Dou, Jun Han, "A Hardware efficient variable length FFT processor for low power application," *IEEE Trans. Very Large Scale Integer. (VLSI) Syst.* Nov 2013.
15. Safwat Mostafa Noor , Eugene John, and Manoj Panday, "Design and Implementation of an Ultralow-Energy FFT ASIC for Processing ECG in Cardiac Pacemakers" *IEEE Trans. Very Large Scale Integer. (VLSI) Syst.* November 12, 2018.